



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**
Membre de **HONORIS UNITED UNIVERSITIES**

Filière : 4ème Année Développement Digital et Système
d'Information

Projet de Fin de Module

Développement d'une Application Mobile Android :
"QI Test" avec Architecture Sécurisée FastAPI &
Supabase

Réalisé par :

Fatim-Zahra Chitaoui

Encadré par :

M. Bousmah

Année Universitaire : 2025 – 2026

Dédicaces

À mes chers parents, pour leur amour inconditionnel, leurs sacrifices et leur soutien permanent tout au long de mon parcours universitaire.

À mes professeurs de l'EMSI, qui m'ont transmis les connaissances et le goût de l'excellence.

À mes ami(e)s et à tous ceux qui ont contribué de près ou de loin à la réalisation de ce travail.

Remerciements

Je tiens tout d'abord à exprimer ma profonde gratitude à mon encadrant, **M. Bousmah**, pour sa disponibilité, ses précieux conseils et ses orientations judicieuses qui m'ont guidée tout au long du développement de ce projet.

Mes remerciements s'adressent également au corps professoral et administratif de l'École Marocaine des Sciences de l'Ingénieur (EMSI) pour la qualité de la formation dispensée et pour leur dévouement constant tout au long de mon parcours de formation.

Je tiens à exprimer ma reconnaissance la plus profonde et toute mon affection à ma famille, qui a constitué mon plus grand pilier. Un immense merci à mes chers parents pour leur amour inconditionnel, leurs sacrifices sans fin et leur soutien indéfectible qui m'ont permis d'en arriver là aujourd'hui. À mes frères, pour leur présence rassurante, leurs encouragements constants et leur bienveillance à chaque étape.

Résumé

Ce projet présente la conception et la réalisation d'une application mobile d'évaluation cognitive sous Android, intitulée "**QI Test**". L'objectif est d'allier l'interactivité d'un quiz de logique à des mécanismes poussés de sécurité et d'intelligence artificielle. Avant d'accéder au test, une vérification biométrique humaine par reconnaissance faciale active (*Liveness Detection* via détection de sourire) est imposée. Pendant le quiz de 30 secondes par question, l'utilisateur bénéficie d'une accessibilité par synthèse vocale tandis qu'une surveillance continue via la caméra frontale détecte les tentatives de triche. À la fin, le système géolocalise l'utilisateur et met à jour un classement compétitif en temps réel . L'architecture repose sur un frontend Android (Java), un backend proxy sécurisé **FastAPI (Python)** pour isoler la logique métier, et une infrastructure Cloud **Supabase** assurant la gestion robuste des données et de l'authentification.

Mots-clés : Android (Java), FastAPI, Supabase, Détection Faciale, Anti-triche, Géolocalisation.

Abstract

This project details the design and implementation of an advanced cognitive assessment Android application named "**QI Test**". The primary objective is to combine interactive logic quizzes with rigorous security protocols and computer vision features. Before initiating the test, a biometric liveness detection mechanism (smile detection) is enforced via the front camera to prevent automated bot access. During the test, which features a 30-second countdown per question, text-to-speech accessibility is available while continuous camera monitoring flags potential cheating behaviors. Upon completion, the user's GPS location is captured to build a geographic real-time . The application is built upon a hybrid architecture consisting of a native Android frontend (Java), a secure proxy backend developed with **FastAPI (Python)** to shield business logic, and a cloud infrastructure powered by **Supabase** for robust user authentication and data management.

Keywords : Android (Java), FastAPI, Supabase, Face Detection, Anti-cheating, Geolocation.

Liste des Abréviations

API	Application Programming Interface
DAO	Data Access Object
EMSI	École Marocaine des Sciences de l'Ingénieur
GPS	Global Positioning System
IHM	Interface Homme-Machine
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVC	Model-View-Controller
QI	Quotient Intellectuel
RGPD	Règlement Général sur la Protection des Données
SDK	Software Development Kit
TTS	Text-To-Speech
UI	User Interface

Table des matières

Dédicaces	i
Remerciements	ii
Résumé	iii
Abstract	iv
Liste des Abréviations	v
Introduction Générale	1
1 Contexte Général du Projet	2
1.1 Présentation du projet	2
1.1.1 Problématique	2
1.1.2 Solution proposée	3
1.2 Méthodologie de travail et planification	3
1.2.1 Diagramme de Gantt	4
2 Analyse et Conception	5
2.1 Etude fonctionnelle	5
2.1.1 Analyse des besoins fonctionnels	5
2.1.2 Analyse des besoins non fonctionnels	6
2.2 Etude Conceptuelle	7
2.2.1 Regles de gestion	7
2.2.2 Diagramme UML	7
Conclusion	13
3 Étude Technique	14
3.1 Architecture du projet	14
3.2 Technologies utilisées	15
3.3 Environnement de travail	15

4	Mise en œuvre et Réalisation	17
4.1	Réalisation et Interfaces graphiques	17
4.1.1	Interface de Connexion	18
4.1.2	Vérification Biométrique Active	19
4.1.3	Accueil et Règles du Test	20
4.1.4	Interface du Quiz et Accessibilité	21
4.1.5	Résultats, Localisation et Rapport de Triche	22
	Conclusion	23
	Conclusion Générale	24

Introduction Générale

Le développement d'applications mobiles modernes a franchi une étape charnière où l'expérience utilisateur doit impérativement s'aligner sur des standards de sécurité et d'intelligence logicielle élevés. Dans un écosystème numérique de plus en plus vulnérable aux fuites de données, la conception d'outils interactifs demande une rigueur architecturale sans faille. C'est dans cette perspective que s'inscrit le présent projet, portant sur la création d'une application Android d'évaluation cognitive intitulée "**QI Test**".

L'enjeu de ce travail est de proposer une solution mobile robuste, capable non seulement de mesurer les facultés intellectuelles des utilisateurs à travers des modules de quiz dynamiques, mais aussi de garantir l'intégrité et la confidentialité des échanges. Pour répondre à ces défis, le projet repose sur trois piliers technologiques majeurs :

- **L'interaction Homme-Machine avancée** : L'intégration de la synthèse vocale (*Text-to-Speech*) offre une immersion accrue, tandis que l'usage de la caméra frontale permet une détection faciale en temps réel, assurant que l'utilisateur est bien présent et actif durant l'évaluation (détection du sourire en phase initiale et détection de présence continue pour l'anti-triche).
- **Une Architecture Backend "Zero-Trust"** : En rupture avec les architectures mobiles simplistes, ce projet introduit un serveur proxy intermédiaire développé avec **FastAPI (Python)**. Ce choix stratégique permet d'isoler la logique métier et de protéger les identifiants de connexion ainsi que les requêtes directes à la base de données **Supabase**, empêchant ainsi toute tentative d'extraction de clés API par ingénierie inverse sur l'application mobile.
- **Gestion des données en temps réel et Géolocalisation** : L'utilisation d'une infrastructure cloud permet une synchronisation fluide des questions et une sauvegarde sécurisée des scores des utilisateurs, couplée à une capture GPS finale pour cartographier le niveau d'intelligence par région.

Ce rapport documente l'intégralité du cycle de vie du projet. Il débute par une analyse rigoureuse des besoins, se poursuit par une description détaillée de la conception de l'architecture sécurisée, et s'achève par la présentation de l'implémentation logicielle ainsi que des tests de validation effectués.

Chapitre 1

Contexte Général du Projet

Introduction

Le succès d'un projet d'ingénierie logicielle dépend d'une compréhension approfondie de son environnement, des défis qu'il cherche à relever et de la rigueur méthodologique appliquée à sa réalisation. Ce premier chapitre pose le cadre contextuel de notre application d'évaluation cognitive intitulée "**QI Test**". Nous y exposerons la genèse du projet, l'analyse des problématiques actuelles liées à la triche et à la sécurité des applications mobiles connectées, ainsi que la solution logicielle que nous avons conçue pour y répondre. Enfin, nous détaillerons la démarche méthodologique adoptée pour orchestrer et planifier efficacement l'ensemble du cycle de développement.

1.1 Présentation du projet

Le projet consiste en la conception et le développement d'une plateforme mobile innovante d'évaluation du quotient intellectuel (QI) couplée à une architecture backend robuste et hautement sécurisée. L'application mobile cible les utilisateurs désireux de tester leurs compétences cognitives à travers des quiz dynamiques et chronométrés. Au-delà du simple aspect ludique ou académique, ce projet vise à repenser l'architecture des applications mobiles classiques en intégrant des modules de vision par ordinateur en temps réel et en adoptant une approche de sécurité centralisée pour protéger l'intégrité des données et de l'infrastructure Cloud.

1.1.1 Problématique

Les solutions d'évaluation et de quiz en ligne actuelles souffrent de deux vulnérabilités majeures qui freinent leur adoption dans un cadre officiel ou hautement compétitif :

1. **La vulnérabilité aux fraudes et l'absence de contrôle d'identité** : La majorité des applications de quiz ne disposent d'aucun mécanisme permettant de valider

que l'utilisateur derrière l'écran est bien un être humain (et non un script automatisé ou un robot) ni de garantir l'absence de triche (aide extérieure, consultation de documents) pendant le déroulement de l'épreuve.

2. **L'insécurité des architectures mobiles ("Reverse Engineering")** : Dans une architecture mobile standard, l'application frontend communique souvent directement avec la base de données cloud ou des API tierces. Un utilisateur malveillant peut facilement décompiler le fichier binaire (APK), extraire les clés API de Supabase ou les identifiants de connexion par ingénierie inverse, et ainsi manipuler les scores, voler des données ou fausser le classement général.

1.1.2 Solution proposée

Pour répondre à ces défis, nous proposons le système **"QI Test"**, une application mobile native Android (Java) adossée à une architecture proxy **"Zero-Trust"** :

- **Authentification Humaine et Anti-triche** : Implémentation d'une vérification biométrique au démarrage exigeant un sourire de l'utilisateur pour valider sa présence physique (*Liveness Detection*). De plus, un flux caméra reste actif en tâche de fond pour superviser le comportement de l'utilisateur pendant les 30 secondes allouées à chaque question.
- **Architecture Proxy Sécurisée avec FastAPI** : Pour pallier les risques d'ingénierie inverse, nous avons développé un serveur backend intermédiaire en **FastAPI (Python)** via un script centralisé `main.py`. L'application mobile ne connaît jamais l'adresse directe ni les clés maîtresses de notre base de données. FastAPI agit comme un bouclier étanche : il intercepte les requêtes du mobile, valide les jetons de session (JWT), extrait de manière sécurisée les questions depuis **Supabase (PostgreSQL)** et renvoie uniquement les données nécessaires au smartphone.
- **Interactivité et Cartographie** : L'application intègre une synthèse vocale (*Text-to-Speech*) pour la lecture des questions afin d'améliorer l'accessibilité, capture les coordonnées GPS à l'issue du test pour générer des statistiques régionales, et maintient un classement dynamique (*Leaderboard*) des utilisateurs les plus performants.

1.2 Méthodologie de travail et planification

La réalisation d'un projet regroupant des technologies aussi diverses (Mobile, Vision par ordinateur, Backend, Cloud) nécessite une organisation rigoureuse pour respecter les délais et garantir la qualité du code produit.

1.2.1 Diagramme de Gantt

La planification temporelle du projet s'est étalée sur plusieurs phases clés, structurant le cycle de vie de l'application depuis la recherche conceptuelle jusqu'au déploiement final :

1. **Phase 1 : Étude de cadrage et spécification des besoins** — Analyse des besoins fonctionnels, spécification technique et modélisation de l'architecture proxy.
2. **Phase 2 : Développement du Frontend Android** — Conception des interfaces graphiques en Java (formulaires d'authentification, interface de quiz), gestion du cycle des questions et intégration du composant caméra.
3. **Phase 3 : Développement du Backend FastAPI & Configuration Cloud** — Écriture des routes API dans `main.py`, implémentation de la logique métier et structuration de la base de données PostgreSQL sur Supabase.
4. **Phase 4 : Intégration de l'IA et Sécurisation** — Implémentation des algorithmes de détection faciale (sourire et anti-triche continue) et sécurisation des routes par jetons JWT.
5. **Phase 5 : Phase de Tests, Validation et Rédaction** — Tests unitaires, tests d'intégration de bout en bout et finalisation du rapport technique.

Note : Le diagramme de Gantt visuel modélisant ce calendrier avec l'enchaînement exact de ces phases et des Sprints associés est illustré dans la suite de ce document.

Conclusion

En somme, ce premier chapitre a permis de poser les fondations de notre projet en explicitant le contexte, les problématiques de sécurité majeures des applications connectées, et l'architecture innovante choisie pour y faire face. La mise en place d'une planification agile claire nous donne une feuille de route précise pour aborder sereinement la phase suivante dédiée à l'analyse fonctionnelle et à la conception technique détaillée du système.

Chapitre 2

Analyse et Conception

Introduction

Après avoir défini le contexte général du projet et planifié rigoureusement ses différentes étapes, il devient indispensable de formaliser les exigences architecturales et opérationnelles du système avant d'entamer sa phase de réalisation logicielle. Ce deuxième chapitre est entièrement consacré à l'analyse approfondie et à la conception technique détaillée de notre application d'évaluation cognitive, baptisée **"QI Test"**. L'objectif de cette étape charnière est de traduire les besoins métiers et les cas d'utilisation en spécifications techniques claires, stables et hautement structurées.

Pour ce faire, nous débuterons par une étude fonctionnelle rigoureuse afin de cerner avec précision les attentes des utilisateurs, les contraintes de performance en temps réel, ainsi que les exigences de sécurité strictes imposées par notre architecture **"Zero-Trust"**. Par la suite, nous modéliserons les interactions et la dynamique du système à travers les différents diagrammes de langage UML, posant ainsi des fondations solides pour garantir la robustesse du backend FastAPI, l'intégrité de la base de données Supabase, et la fluidité de l'application mobile Android face aux contraintes d'intelligence artificielle embarquée.

2.1 Etude fonctionnelle

L'étude fonctionnelle permet de définir le périmètre d'action de l'application en décrivant de manière structurée les interactions entre l'utilisateur et le système. Elle offre une vision claire du comportement attendu de la plateforme à travers ses différentes composantes techniques.

2.1.1 Analyse des besoins fonctionnels

Les besoins fonctionnels décrivent les actions directes que l'application doit être capable d'exécuter. Pour notre système, ils s'articulent autour des axes principaux suivants :

- **Vérification biométrique humaine (Liveness Detection) :** Le système doit obligatoirement activer la caméra frontale au démarrage et demander explicitement à l'utilisateur de sourire afin de valider sa présence physique et bloquer les accès automatisés.
- **Gestion des comptes utilisateurs :** L'application doit proposer un formulaire de connexion sécurisé ainsi qu'un lien de redirection vers une interface d'inscription pour les nouveaux utilisateurs, le tout adossé au service d'authentification.
- **Déroulement dynamique du quiz :** Après connexion, le système doit charger les questions de logique et imposer une contrainte de temps stricte de 30 secondes par question avec un compte à rebours interactif.
- **Options d'accessibilité vocale :** L'utilisateur doit avoir la possibilité d'activer à tout moment une option de synthèse vocale pour qu'une voix assistée lise à haute voix l'énoncé des questions.
- **Surveillance continue et anti-triche :** La caméra frontale doit rester active tout au long du test pour surveiller le comportement de l'utilisateur et détecter d'éventuelles anomalies ou tentatives de triche.
- **Géolocalisation et Classement en temps réel :** À la fin du test, l'application doit capturer la position GPS de l'utilisateur, lier cette coordonnée à son score final et mettre à jour le classement général mondial pour afficher la répartition géographique des scores les plus élevés.

2.1.2 Analyse des besoins non fonctionnels

Les besoins non fonctionnels définissent les exigences de qualité, de performance et de sécurité qui encadrent le fonctionnement des fonctionnalités décrites précédemment :

- **Sécurité et Confidentialité (Zero-Trust) :** Aucune clé API maîtresse ou identifiant direct de la base de données ne doit transiter ou être stocké sur le smartphone. Toutes les requêtes sensibles doivent être interceptées et validées par le serveur intermédiaire FastAPI via des jetons JWT sécurisés.
- **Performance et Disponibilité :** Le serveur backend FastAPI et la base de données Supabase doivent garantir un temps de réponse minimal (inférieur à quelques millisecondes) pour le chargement des questions et la mise à jour du classement.
- **Ergonomie et Fluidité :** L'interface utilisateur Android doit être intuitive, réactive et offrir une navigation fluide, notamment lors des transitions d'écrans et de la gestion du chronomètre en temps réel.
- **Robustesse et Tolérance aux pannes :** L'application doit être capable de gérer proprement les interruptions de réseau ou les pertes de signal GPS sans faire planter l'application ni altérer les données locales de l'utilisateur.

2.2 Etude Conceptuelle

2.2.1 Regles de gestion

2.2.2 Diagramme UML

Cas d'utilisation

Le diagramme de cas d'utilisation constitue la première étape de notre modélisation UML. Il permet de structurer les fonctionnalités du système en identifiant les interactions entre les acteurs externes et les différents modules de l'application.

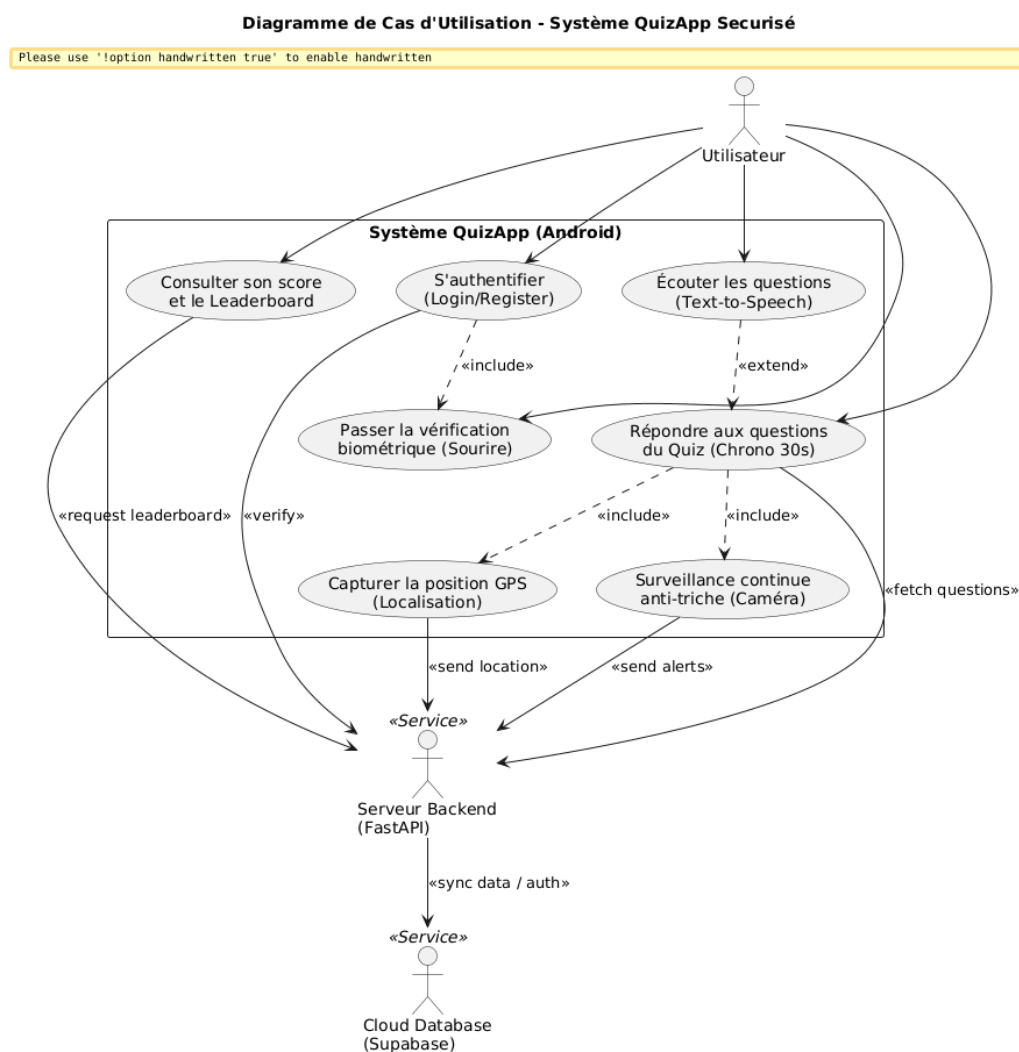


FIGURE 2.1 – Diagramme de cas d'utilisation général du système QI Test

L'analyse de ce diagramme met en évidence un acteur principal (l'Utilisateur) au sommet de la hiérarchie et deux acteurs secondaires agissant sous forme de services connectés : le Serveur Backend (FastAPI) et la base de données Cloud (Supabase). Le cycle opérationnel se décompose ainsi :

- **Contrôle d'accès et cycle d'authentification** : Pour accéder à l'application, l'utilisateur sollicite le cas d'utilisation "S'authentifier (Login/Register)". Ce processus inclut obligatoirement («*include*») l'étape de sécurité "Passer la vérification biométrique (Sourire)". La requête d'authentification finale est transmise via la liaison «*verify*» au serveur FastAPI qui valide ou rejette la demande.
- **Déroulement du Quiz et modularité fonctionnelle** : Lorsque l'utilisateur initie l'action "Répondre aux questions du Quiz (Chrono 30s)", l'application mobile récupère dynamiquement les données («*fetch questions*») auprès du backend. Ce cas d'utilisation central englobe obligatoirement deux sous-systèmes critiques :
 - **"Surveillance continue anti-triche (Caméra)"** : un processus d'arrière-plan qui analyse le comportement de l'utilisateur et communique directement les anomalies constatées au serveur («*send alerts*»).
 - **"Capturer la position GPS (Localisation)"** : un module déclenché à la clôture du test pour transmettre les coordonnées géographiques au serveur («*send location*»).

De plus, le cas d'utilisation "Écouter les questions (Text-to-Speech)" vient étendre («*extend*») facultativement l'expérience utilisateur pour offrir une accessibilité vocale à la demande.
- **Exploitation des résultats et compétitivité** : L'utilisateur peut à tout moment solliciter le cas d'utilisation "Consulter son score et le Leaderboard". L'application mobile formule alors une requête («*request leaderboard*») auprès de l'API afin d'afficher le classement mis à jour en fonction des scores et de la répartition géographique des participants.
- **Architecture Proxy et Persistance des données** : Le diagramme illustre parfaitement l'étanchéité de l'approche Zero-Trust. L'application mobile Android n'interagit jamais directement avec le stockage cloud. Le **Serveur Backend (FastAPI)** centralise et filtre l'ensemble des flux d'alertes, de localisation et de scores, puis se charge de synchroniser de manière isolée et sécurisée les informations («*sync data / auth*») avec la base **Cloud Database (Supabase)**.

Diagramme de séquence

Le diagramme de séquence permet d'étudier la dynamique du système en représentant l'enchaînement chronologique des messages et des requêtes échangés entre l'utilisateur, l'application mobile Android et les services de l'architecture backend lors d'un cycle complet d'évaluation.

Le scénario modélisé par la Figure 2.2 est structuré en trois phases majeures :

1. Étape 1 : Authentification & Vérification Biométrique

- L'utilisateur initie le processus en soumettant ses identifiants depuis l'interface

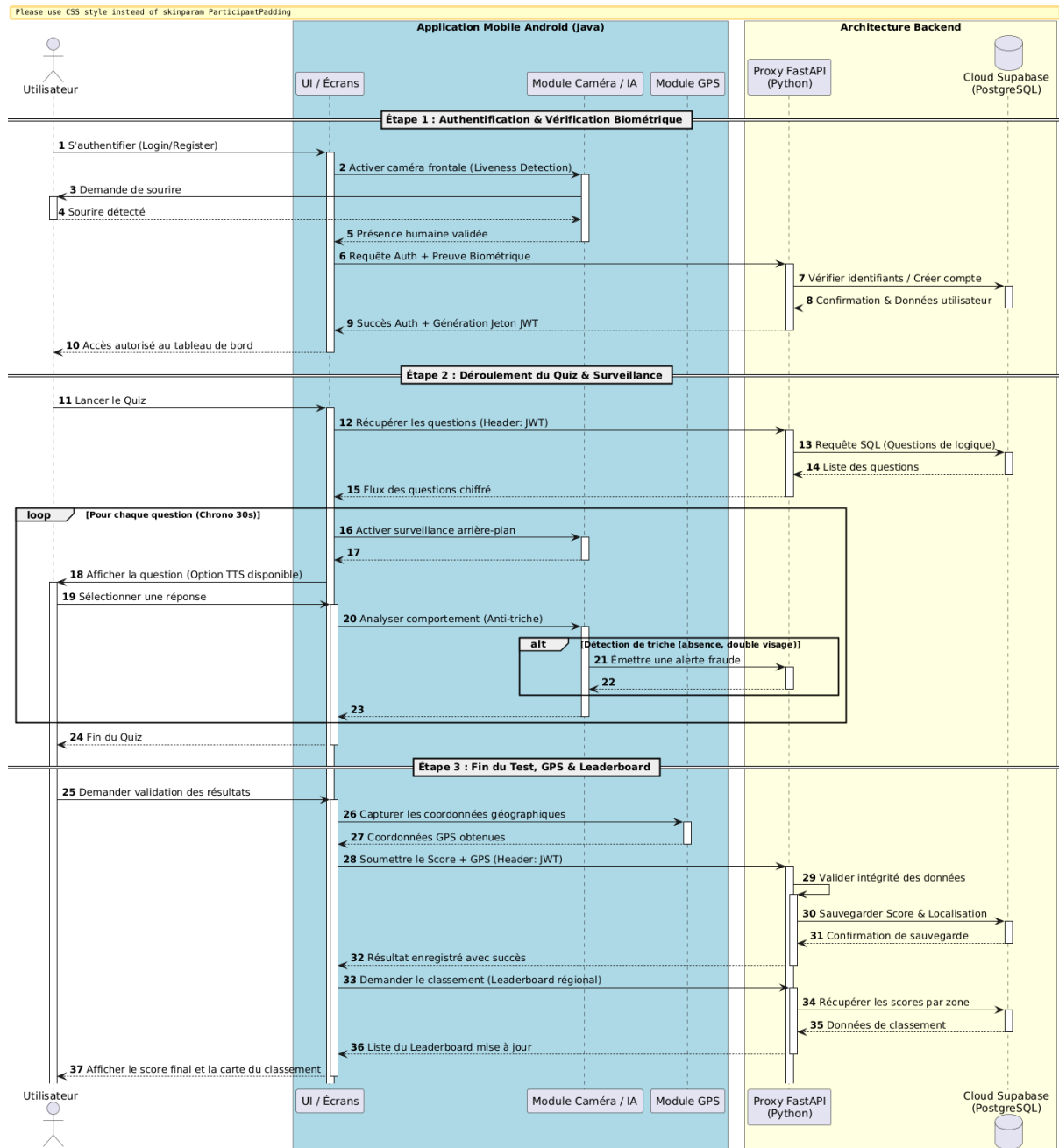


FIGURE 2.2 – Diagramme de séquence global du système QuizApp Sécurisé

(Message 1). L'IHM ordonne alors au module caméra d'exécuter la vérification de présence humaine active (*Liveness Detection*) (Message 2).

- Le système exige un sourire de l'utilisateur (Message 3). Une fois ce dernier détecté et validé (Messages 4 et 5), l'IHM envoie une requête groupée incluant la preuve biométrique au serveur proxy FastAPI (Message 6).
- FastAPI intercepte la requête, communique de manière isolée avec la base Supabase pour créer ou vérifier le compte (Messages 7 et 8), puis génère un jeton de session sécurisé JWT qu'il renvoie à l'application mobile (Message 9), autorisant ainsi l'accès au tableau de bord (Message 10).

2. Étape 2 : Déroulement du Quiz & Surveillance

- Lorsque l'utilisateur lance le quiz (Message 11), l'application mobile transmet une requête HTTP à FastAPI pour récupérer les questions en joignant le jeton JWT dans le Header (Message 12). FastAPI exécute une requête SQL sécurisée auprès de Supabase (Messages 13 et 14) et retourne un flux de questions chiffré à l'interface (Message 15).
- À l'intérieur d'une boucle itérative (*loop*) pour chaque question chronométrée à 30 secondes, l'IHM active la surveillance vidéo en arrière-plan (Messages 16 et 17) et affiche la question (Message 18).
- Pendant que l'utilisateur sélectionne sa réponse (Message 19), le module IA analyse en continu son comportement (Message 20). En cas d'anomalie (absence, présence d'un tiers), un bloc alternatif (*alt*) est déclenché pour émettre instantanément une alerte de fraude vers le serveur FastAPI (Messages 21, 22 et 23). La phase prend fin à l'expiration du quiz (Message 24).

3. Étape 3 : Fin du Test, GPS & Leaderboard

- Dès que l'utilisateur demande la validation de ses résultats (Message 25), l'application mobile sollicite le module GPS embarqué pour capturer ses coordonnées géographiques en temps réel (Messages 26 et 27).
- L'IHM transmet ensuite le score final combiné à la localisation géographique au proxy FastAPI (Message 28). Ce dernier applique des règles de validation pour contrôler l'intégrité des flux (Message 29) avant d'ordonner la persistance des données sur Supabase (Messages 30 et 31), puis confirme la bonne sauvegarde à l'appareil (Message 32).
- Enfin, une dernière requête est émise pour récupérer le classement régional (*Leaderboard*) (Message 33). FastAPI extrait les données agrégées par zone géographique auprès de Supabase (Messages 34 et 35) et retourne la liste mise à jour (Message 36), permettant à l'interface mobile d'afficher le score final de l'utilisateur ainsi qu'une cartographie interactive du classement (Message 37).

Diagramme d'activité

Le diagramme d'activité permet de modéliser le comportement cinématique du système en décrivant les flux de contrôle opérationnels, les structures de décision et les traitements parallèles qui régissent l'application.

Le processus représenté par la Figure 2.3 débute par le lancement de l'application mobile et l'affichage de l'écran d'authentification, avant de s'articuler autour de trois étapes majeures cloisonnées :

1. Étape 1 : Contrôle Biométrique & Accès

- Après la saisie des identifiants de connexion, le système active la caméra frontale pour effectuer une détection de présence humaine active (*Liveness Detection*) en demandant un sourire à l'utilisateur.
- Une structure conditionnelle évalue la réaction : si aucun sourire n'est détecté dans le temps imparti, l'accès est refusé et le flux s'interrompt. Si le sourire est détecté, la requête d'authentification accompagnée de la preuve biométrique est transmise au proxy FastAPI.
- Le backend interroge alors Supabase pour valider les identifiants. Si ces derniers sont invalides, une erreur est retournée. Si la vérification réussit, le système génère un jeton de session JWT et redirige l'utilisateur vers son tableau de bord.

2. Étape 2 : Évaluation & Surveillance Continue

- Cette étape s'initie par le lancement du quiz de logique, suivi d'une requête de récupération des questions via FastAPI pour charger et initialiser le flux applicatif.
- Une barre de synchronisation (*Fork*) sépare ensuite l'exécution en deux threads parallèles :
 - **Le thread de gauche (Flux applicatif principal)** : Il démarre le chronomètre de 30 secondes par question. Une boucle affiche la question courante, vérifie si l'option *Text-to-Speech* est activée pour lire l'énoncé à haute voix, et attend la saisie de l'utilisateur. La boucle se répète tant qu'une question suivante est disponible et que le temps global n'est pas écoulé.
 - **Le thread de droite (Surveillance IA)** : Il active en arrière-plan le module caméra pour analyser en continu le flux vidéo. Si un comportement suspect ou une absence est détectée, le système émet instantanément une alerte de fraude au serveur FastAPI et journalise l'incident en appliquant une pénalité. Ce contrôle se répète en boucle tant que le quiz est en cours d'exécution.
- Une fois le quiz terminé, les deux branches convergent vers une barre de jointure (*Join*) pour clôturer officiellement la session de test.

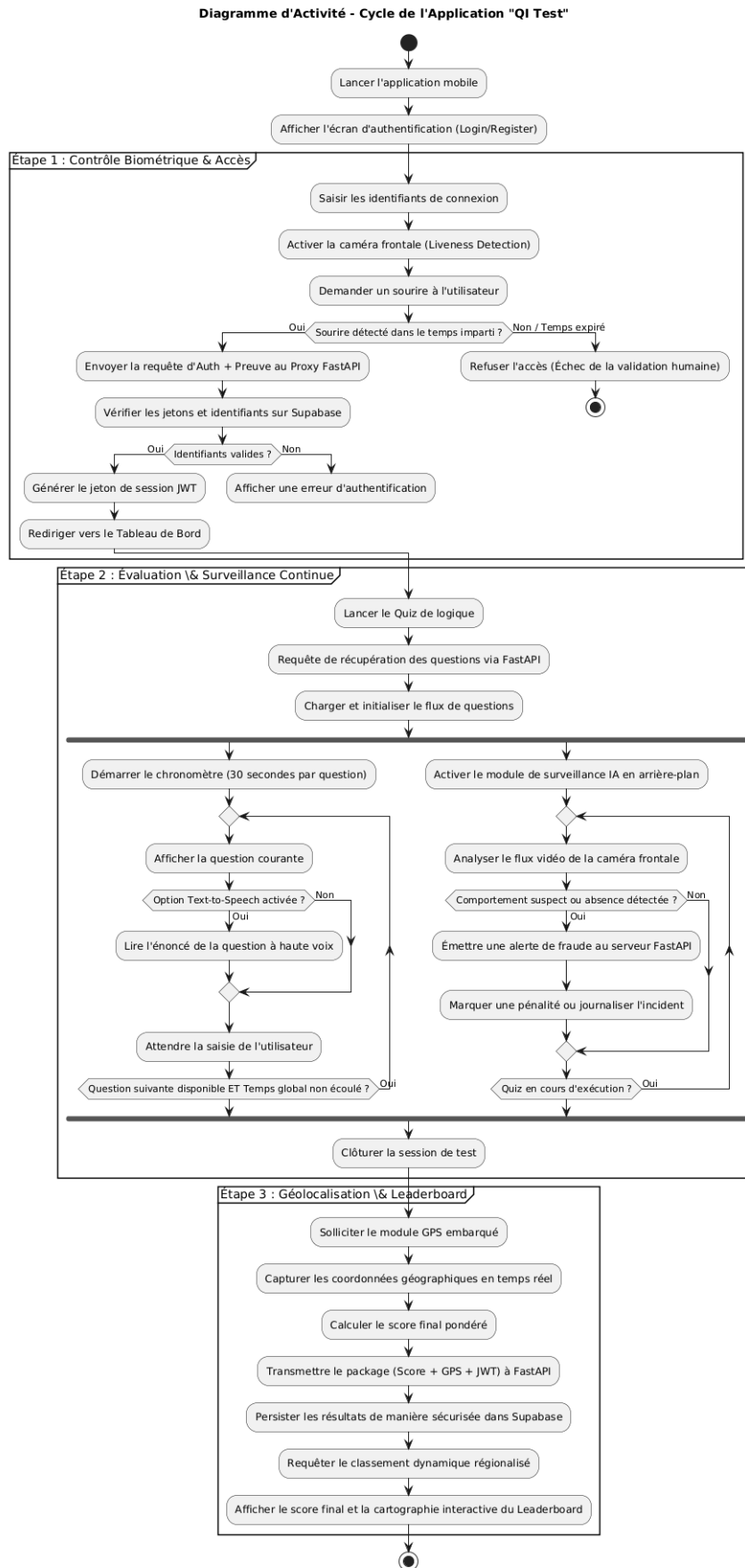


FIGURE 2.3 – Diagramme d'activité du cycle de vie de l'application QI Test

3. Étape 3 : Géolocalisation & Leaderboard

- Dès la clôture du test, l'application sollicite le module GPS embarqué afin de capturer les coordonnées géographiques de l'utilisateur en temps réel.
- Le système calcule le score final pondéré, puis transmet l'ensemble des données (Score, GPS) au serveur FastAPI.
- Le proxy sécurise la transaction et persiste les résultats dans la base Supabase. Enfin, l'application effectue une dernière requête pour récupérer le classement dynamique régionalisé afin d'afficher le score final de l'utilisateur ainsi qu'une cartographie interactive du *Leaderboard*.

Conclusion

En somme, ce deuxième chapitre a permis de mener à bien l'étape cruciale d'analyse et de conception technique de l'application "QI Test". À travers une expression rigoureuse des besoins fonctionnels et non fonctionnels, nous avons défini un périmètre clair pour notre système, plaçant la sécurité "Zero-Trust" et la surveillance en temps réel au cœur de nos priorités.

La modélisation UML s'est avérée fondamentale pour structurer cette vision. Le diagramme de cas d'utilisation a posé les frontières fonctionnelles du système, tandis que les diagrammes de séquence et d'activité ont traduit la dynamique et la cinématique des flux, mettant en lumière le parallélisme entre l'évaluation cognitive et la surveillance antitriche par vision par ordinateur. Enfin, le diagramme de classes a formalisé l'architecture statique multi-tiers, matérialisant le découpage étanche entre l'IHM Android (Java), le proxy intermédiaire FastAPI (Python) et la persistance sur le Cloud Supabase.

Forts de ces spécifications techniques et de ces modèles architecturaux stables, nous disposons désormais de toutes les fondations nécessaires pour aborder sereinement le chapitre suivant, dédié à la réalisation logicielle et à l'implémentation concrète de ces différents composants.

Chapitre 3

Étude Technique

Introduction

La phase de conception ayant permis de stabiliser les structures logiques et la dynamique des flux de notre système, il est désormais nécessaire de définir l'environnement technique propice à sa matérialisation logicielle. Ce troisième chapitre est consacré à l'étude technique détaillée de la plateforme "QI Test". Nous y exposerons l'architecture globale du projet, qui concrétise l'approche proxy protectrice pour isoler nos données cloud. Par la suite, nous justifierons le choix des différentes technologies, frameworks et bibliothèques mis en œuvre pour relever les défis de la vision par ordinateur, de la sécurité et de la géolocalisation. Enfin, nous détaillerons l'environnement de travail, comprenant les outils de développement et de gestion de projet qui ont structuré notre flux de production.

3.1 Architecture du projet

Pour garantir une étanchéité totale et contrer les vulnérabilités liées à l'ingénierie inverse courantes sur les plateformes mobiles connectées, ce projet rejette l'architecture standard à deux niveaux au profit d'une **Architecture Multi-tiers (3-Tier)** basée sur un **Proxy Zero-Trust**.

Cette configuration répartit les responsabilités en trois couches isolées :

- **La couche Présentation (Client Mobile)** : Représentée par l'application native Android, elle prend en charge l'affichage des interfaces graphiques, la capture des flux de la caméra frontale pour l'analyse de triche, et la collecte des coordonnées GPS. Elle ne communique jamais directement avec le stockage cloud et ne possède aucune clé maîtresse de base de données.
- **La couche Métier (Proxy Backend FastAPI)** : Positionnée comme un intermédiaire étanche, cette passerelle centralise la logique de l'application. Elle intercepte les requêtes HTTP de l'application mobile, traite les alertes asynchrones de

triche et applique les règles de calcul des scores.

- **La couche Données (Serveur Cloud Supabase)** : Dédiée exclusivement à la persistance et à la sécurité de l'infrastructure relationnelle PostgreSQL, elle ne répond qu'aux requêtes provenant directement de l'adresse IP sécurisée du serveur FastAPI, protégeant ainsi l'intégrité des questions de logique et des comptes utilisateurs.

3.2 Technologies utilisées

La concrétisation de notre système a nécessité la sélection de technologies complémentaires, alliant performance d'exécution, intégration d'algorithmes d'intelligence artificielle et sécurité des protocoles de communication :

- **Android Studio & Java** : Choisi pour le développement natif de la couche de présentation. Java offre un contrôle total sur les composants matériels du smartphone, permettant une gestion fine des cycles de vie des activités, du module de géolocalisation (*FusedLocationProviderClient*) et de l'interactivité du chronomètre de 30 secondes.
- **FastAPI (Python)** : Retenu pour l'implémentation du serveur proxy en raison de ses performances de rapidité exceptionnelles et de son support natif des opérations asynchrones. Basé sur ASGI, il permet de traiter simultanément et sans blocage les requêtes de réponses aux quiz et les flux d'alertes de fraude émis par les smartphones.
- **Supabase (PostgreSQL)** : Utilisé comme solution de Backend-as-a-Service (BaaS) pour héberger notre base de données relationnelle. PostgreSQL garantit le respect des propriétés ACID pour la sauvegarde des scores géolocalisés, tandis que Supabase simplifie la gestion des accès via l'authentification centralisée.
- **Outils de Vision par Ordinateur (Android Face Detection API)** : Intégrés au sein du frontend mobile pour exécuter la vérification biométrique (*Liveness Detection* par détection du sourire) et la surveillance continue, garantissant le traitement de l'image directement en local pour préserver la bande passante.
- **Android Text-to-Speech (TTS)** : API native exploitée pour assurer l'accessibilité de l'application en convertissant les chaînes de caractères des questions de logique en flux audio clair à destination de l'utilisateur.

3.3 Environnement de travail

Le développement de la plateforme s'est appuyé sur une suite d'outils logiciels et de plateformes collaboratives visant à optimiser la qualité du code et à structurer le calendrier de réalisation :

- **Environnements de Développement Intégrés (IDE) :** **Android Studio** a été exclusivement utilisé pour compiler, déboguer et profiler l'application mobile en Java. En parallèle, **Visual Studio Code** a servi d'environnement pour l'écriture des scripts de routage et de logique métier du backend en Python.
- **Gestionnaire de versions (Git & GitHub) :** **Git** a permis d'assurer le suivi des modifications du code source à travers un système de branches rigoureux. Le dépôt distant sur **GitHub** a sécurisé la sauvegarde du projet et facilité l'intégration continue des différents modules applicatifs.
- **Postman :** Cet outil a joué un rôle clé pour tester, simuler et valider le comportement des routes d'API exposées par FastAPI (Authentification, récupération des questions, soumission des scores) avant leur intégration finale dans le code Java d'Android.
- **Gestion de projet (Scrum & Jira) :** Pour matérialiser notre démarche agile, la plateforme **Jira** a été exploitée afin de découper le projet en User Stories, de configurer le **Product Backlog** et de suivre l'état d'avancement des différents Sprints hebdomadaires.

Chapitre 4

Mise en œuvre et Réalisation

Introduction

Après avoir validé l'architecture multi-tiers et sélectionné les technologies adaptées, nous abordons la phase de matérialisation logicielle. Ce quatrième chapitre présente la mise en œuvre pratique de l'application "QI Test". Nous y exposerons les interfaces finales de l'application mobile Android, illustrant le parcours utilisateur depuis l'accès sécurisé par biométrie jusqu'à la restitution des résultats géolocalisés.

4.1 Réalisation et Interfaces graphiques

Cette section présente les captures d'écran réelles de l'application, démontrant l'intégration des modules de sécurité, d'intelligence artificielle et de géolocalisation.

4.1.1 Interface de Connexion

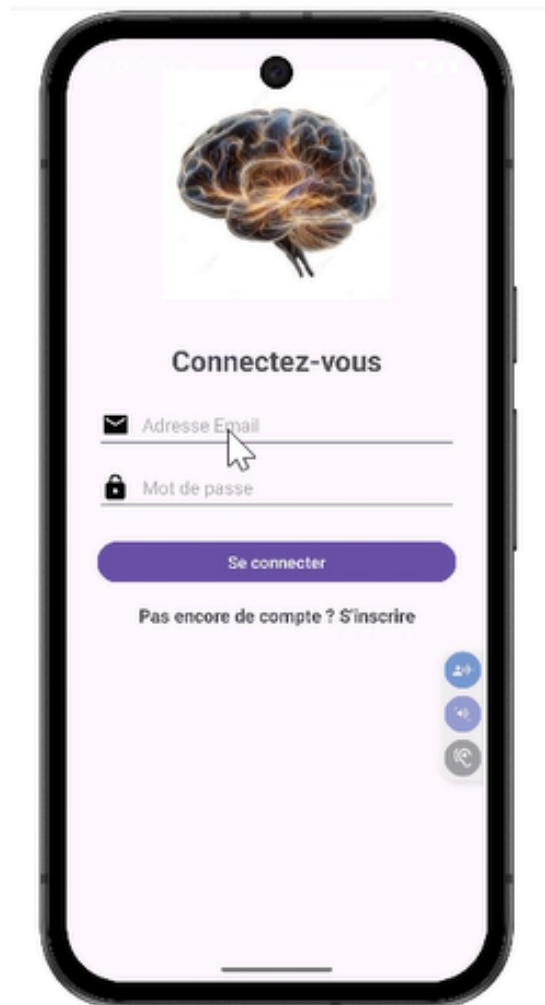


FIGURE 4.1 – Écran de connexion

L'interface de la Figure 4.1 permet l'accès sécurisé des utilisateurs préalablement enregistrés via Supabase. Les champs de saisie filtrent les entrées avant de transmettre les requêtes chiffrées vers notre proxy backend FastAPI.

4.1.2 Vérification Biométrique Active



FIGURE 4.2 – Interface de Vérification Biométrique

Comme l'illustre la Figure 4.2, dès la validation des identifiants, le système dresse une barrière de sécurité par reconnaissance faciale où l'utilisateur doit impérativement sourire. Cette étape de *Liveness Detection* permet de bloquer les connexions automatisées par script et de certifier la présence physique d'un être humain.

4.1.3 Accueil et Règles du Test

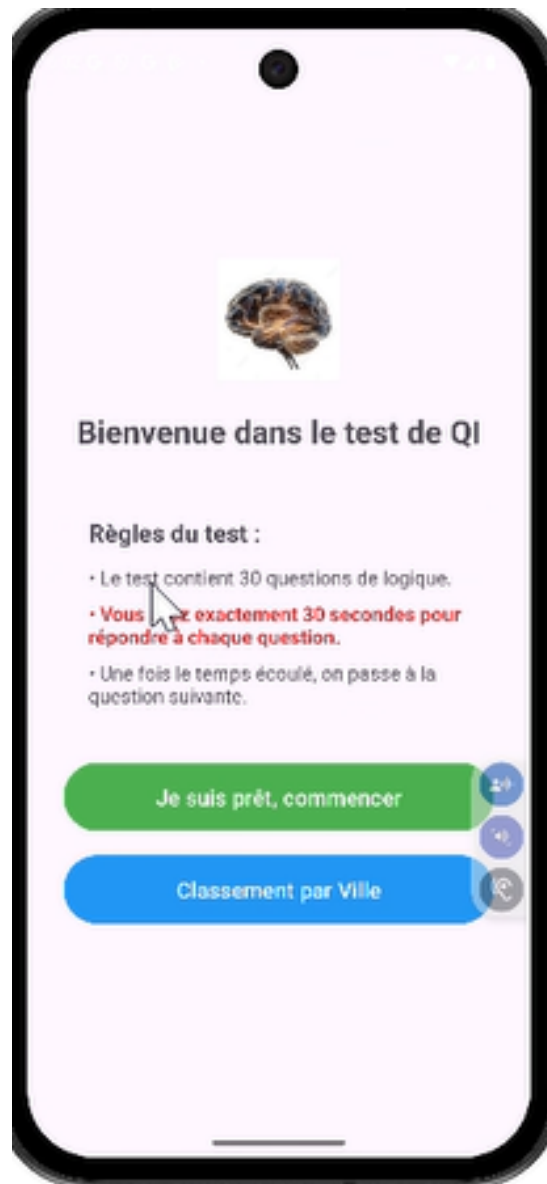


FIGURE 4.3 – Accueil et instructions du test

Comme illustré sur la Figure 4.3, l'utilisateur prend connaissance des modalités avant de commencer l'épreuve : un test contenant un ensemble de 30 questions de logique avec une contrainte de 30 secondes pour répondre à chaque question. Cette interface fournit également un point d'accès direct pour consulter le classement général par ville.

4.1.4 Interface du Quiz et Accessibilité



FIGURE 4.4 – Interface de passation du Quiz

La Figure 4.4 montre une question de logique typique lors du test. On y distingue plusieurs éléments clés : le chronomètre en haut à droite indiquant le temps restant, l'image de l'énigme à résoudre, et le bouton « **Lire à voix haute** » situé en bas qui active la synthèse vocale (TTS). Durant toute cette phase, la caméra frontale analyse en continu le comportement de l'utilisateur.

4.1.5 Résultats, Localisation et Rapport de Triche

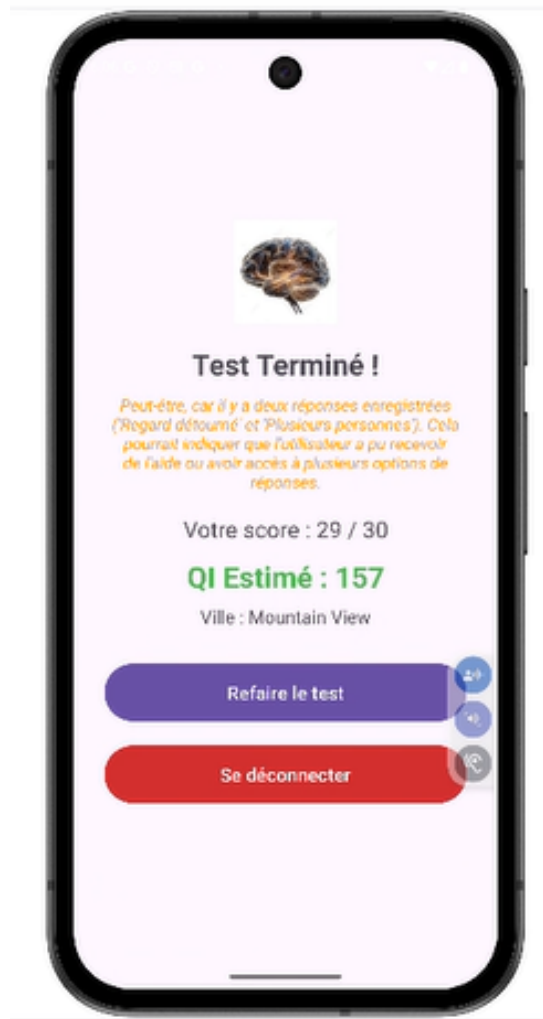


FIGURE 4.5 – Bilan final et analyse de triche

La Figure 4.5 présente l'écran de fin de test généré lors de la clôture de la session. On y observe la consolidation complète des données métiers et de sécurité :

- **Le Score et le QI** : Le calcul automatique basé sur les bonnes réponses fournies (29 / 30) associé à une estimation du QI (157).
- **La Géolocalisation** : La ville de l'utilisateur (Mountain View) récupérée de manière logicielle via le capteur GPS embarqué.
- **Le Rapport de surveillance** : Un message d'alerte contextuel indique si des anomalies ou des comportements suspects ont été enregistrés (*Regard détourné* et *Plusieurs personnes*), validant ainsi le fonctionnement de notre module d'analyse anti-triche par vision artificielle.

Conclusion

En somme, ce quatrième chapitre a matérialisé la convergence entre les exigences conceptuelles de notre cahier des charges et les impératifs de l'implémentation logicielle. La présentation des différentes interfaces graphiques de l'application « QI Test » témoigne non seulement d'un effort d'ergonomie et de fluidité dans le parcours utilisateur, mais valide surtout la viabilité opérationnelle de nos choix architecturaux.

La mise en œuvre conjointe des modules natifs Java sur Android et des services du proxy backend FastAPI a permis de concrétiser avec succès des fonctionnalités à forte complexité technique. Nous avons ainsi pu valider :

- L'efficacité du contrôle biométrique initial (*Liveness Detection*) par détection de sourire, barrant l'accès aux connexions automatisées ;
- L'interactivité et l'accessibilité du quiz grâce à l'intégration de la synthèse vocale (TTS) et d'un chronométrage précis ;
- La robustesse du moteur de surveillance anti-triche basé sur la vision artificielle, capable d'analyser en arrière-plan les comportements suspects et de notifier de manière asynchrone le serveur backend ;
- La précision du suivi géographique via le capteur GPS pour la consolidation d'un leaderboard régionalisé persistant sur Supabase.

En définitive, l'étanchéité de cette architecture multi-tiers et le chiffrement des flux protègent efficacement l'application contre les risques de falsification des scores et d'ingénierie inverse. Ce jalon pratique clôture avec succès le cycle de développement de notre plateforme, nous permettant ainsi de dresser le bilan global de ce projet de fin d'études dans la conclusion générale de ce rapport.

Conclusion Générale

Arrivé au terme de ce projet de fin de module dédié à la conception et à la réalisation de l'application mobile sécurisée d'évaluation cognitive « QI Test », il est temps d'en dresser le bilan global.

L'objectif majeur consistait à bâtir un système d'évaluation en ligne robuste capable de garantir l'intégrité des résultats face aux risques de fraude applicative. Pour y répondre, j'ai pu mené une démarche d'ingénierie rigoureuse soutenue par une modélisation UML approfondie. En adoptant une architecture multi-tiers articulée autour d'un proxy intermédiaire FastAPI, nous avons appliqué les principes de sécurité « Zero-Trust » pour isoler totalement la base de données cloud Supabase des requêtes directes du client mobile. De plus, l'intégration locale d'algorithmes de vision par ordinateur a permis de valider l'accès par contrôle biométrique (*Liveness Detection* par détection de sourire) et d'assurer une surveillance anti-triche continue et non intrusive en arrière-plan. Enfin, l'exploitation conjointe du capteur GPS pour la régionalisation des scores et de l'API de synthèse vocale (TTS) a enrichi l'interactivité de la plateforme via son leaderboard dynamique.

Sur le plan personnel et professionnel, ce projet s'est révélé particulièrement formateur. Il m'a permis de consolider mes compétences techniques sur des technologies modernes et hautement demandées, telles que le développement natif Android (Java), l'asynchronisme en Python (FastAPI), et la gestion de bases de données cloud (Supabase/PostgreSQL). Par ailleurs, la conduite de ce projet suivant la méthodologie agile Scrum à l'aide de l'outil Jira m'a confrontée aux réalités du cycle de production logicielle, renforçant mon autonomie et ma rigueur méthodologique.

Bien que la solution actuelle réponde pleinement aux exigences du cahier des charges initial, elle ouvre des perspectives d'évolution stimulantes pour l'avenir. À court terme, il serait intéressant d'intégrer des modèles de *Deep Learning* plus poussés pour analyser les micro-mouvements oculaires afin d'affiner la détection de triche.